Load-Aware A* Path Planning for Quadruped Robots

Day 23 of 30 · · Week 4: Results Review & Integration · Member 1 — Results Section Input

NAME: HARINI P (Department of Computer Science and Engineering)

1. Nature of Work Done

Day 23 is the second advisory day of Week 4. Member 1's role today is analytical: to deliver deep algorithm-specific insights, flag the highest-impact results for the paper, cross-validate result interpretations from Member 4's Section V draft, and certify technical accuracy across all equations, parameters, and claims. The four tasks below represent the complete Day 23 scope.

Task	Scope	Output	Status
Task 1	Algorithm-Specific Insights	Insight register with algorithmic evidence	Complete ✔
Task 2	Highlight Recommendations for Paper	Priority highlights list for Abstract/Conclusion	Complete ✔
Task 3	Data Interpretation Review	Interpretation guide with scenario-level commentary	Complete ✔
Task 4	Technical Accuracy Certification	Accuracy sign-off table across Sections III–V	Complete ✔

2. Algorithm-Specific Insights

The following insights are grounded in the mathematical structure of Load-Aware A* as defined in Methodology v2.0 (Sections III-C through III-E). Each insight identifies a behaviour that is algorithmically guaranteed — not merely observed — and must be reflected accurately in the paper.

2.1 The S_map Pre-computation Advantage

One of the most operationally significant design choices in Load-Aware A* is the pre-computation of the stability penalty map S_map before the search begins. This has three concrete algorithmic consequences:

- O(1) stability lookup per node expansion: Without S_map, each node expansion would require computing S(n) from scratch using Eq. 4–5 — involving a vector norm calculation over foot positions. With S_map, the penalty is a single dictionary lookup, reducing per-node overhead to negligible levels.
- Decoupling physical computation from search: The CoM displacement model (Eq. 4) and stability penalty (Eq. 5) are computed exactly once per scenario. The search algorithm never re-evaluates the physical model, making Load-Aware A* as computationally lean as Standard A* during the actual path search.
- Deterministic penalty landscape: Because S_map is static (payload position and mass are fixed per scenario), the stability penalty at any cell is constant across all algorithm iterations. This

means the search space has no dynamic uncertainty, and the algorithm's optimality guarantees hold without reservation.

Algorithmic Implication: dT% (compute overhead) should be close to zero for most scenarios. Any scenario with dT% > 5% is attributable to tie-breaking on equal-cost nodes (when two paths have the same f-value) — not to S_map lookup overhead.

2.2 Continuous vs Binary Stability Penalty — Why It Matters

A critical design decision in Load-Aware A* is the use of a continuous graded penalty S(n) rather than a binary constraint (e.g. blocking all cells where M(n) ≤ 0.05 m). The algorithmic consequences of this choice are:

Design Choice	Binary Constraint Approach	Continuous Penalty (Load-Aware A*)
Cell treatment	Cells below M threshold treated as obstacles	All free cells remain traversable; penalty is proportional to instability
Path completeness	May produce no path if all routes pass through low-margin cells	Always finds a path if one geometrically exists — completeness preserved
Optimality	Optimal w.r.t. length among safe-only cells	Optimal w.r.t. combined f(n) = g + αS + h across all cells
Behaviour in tight grids	May fail entirely in high-obstacle-density scenarios	Gracefully routes through least-unstable cells when necessary
S_avg impact	Cannot improve S_avg for forced passages	Minimises cumulative S(n) across the entire path, not just endpoints

Key Insight: The paper must explicitly state that continuity of S(n) is a deliberate design choice that preserves completeness. The phrase 'continuous, graded penalty' should appear in both the Abstract and Section IV.

2.3 The g(n) Accumulation Mechanism and Path-Level Stability

Load-Aware A* accumulates the stability penalty exclusively in g(n) via Eq. 11: g(n_next) = g(n_cur) + 1 + α·S(n_next). This has a specific and important consequence for path-level behaviour:

- Stability penalties compound along the path: A path that passes through three moderately unstable cells (S = 0.15 each) accumulates a g-penalty of 0.45, which may cause the algorithm to prefer a slightly longer path through stable cells with total S-penalty of 0.10.
- Early vs late penalties are weighted equally: Because g(n) accumulates uniformly, a high-S cell near the start of the path is penalised identically to a high-S cell near the goal. This is physically correct — tip-over risk is constant regardless of position along the route.
- The heuristic h(n) is unaffected: Since S(n) is never added to h(n), the Manhattan heuristic remains a lower bound on the true remaining cost-to-goal (which contains only step costs). This is the formal basis of the admissibility proof from Day 20.

2.4 Backwards Compatibility and the α = 0 Degenerate Case

When α = 0, Eq. 10 reduces exactly to f(n) = g(n) + h(n) — Standard A*. This is not merely a mathematical curiosity; it has the following implications for result validation:

- Any scenario where α = 0 results are generated must be identical to Standard A* results: same path, same L, same N, same T. If they differ, there is a code implementation error.
- The α = 0 equivalence provides a built-in regression test for the implementation: running Load-Aware A* with α = 0 should reproduce the Standard A* baseline exactly.
- This backwards compatibility is a formal novelty claim (N3 in Methodology v2.0) and should be explicitly noted in Section IV.

2.5 Node Expansion Behaviour — Why N May Increase or Decrease

The number of nodes expanded (N) by Load-Aware A* compared to Standard A* is not monotonic — it can increase or decrease depending on the stability gradient of the scenario grid:

Scenario Condition	Expected N(LA) vs N(Std)	Algorithmic Reason
Clear stability gradient (high-S zones are spatially clustered)	N(LA) ≈ N(Std) or slightly lower	S_map guides search away from high-S regions early, reducing backtracking
Diffuse stability gradient (S(n) varies slowly across grid)	N(LA) ≥ N(Std)	More nodes explored before algorithm commits to a route through lower-S cells
Payload near path centroid (unavoidable high-S zone)	N(LA) >> N(Std)	Algorithm exhausts alternatives before accepting high-S passage; large open list
Low payload (mp = 3 kg, near-zero S(n))	N(LA) ≈ N(Std)	S_map values near zero; cost function effectively identical to Standard A*

3. Highlight Recommendations — What to Emphasise in the Paper

Based on the algorithmic analysis above and the complete Day 17–22 body of work, the following results and claims carry the highest evidential weight and must be foregrounded in the Abstract, Section IV, and Conclusion.

3.1 Priority Highlights — Ranked by Impact

Priority	Highlight	Where to Feature	Supporting Evidence
P1 — Critical	M_min(LA) > 0.05 m vs M_min(Std) ≤ 0.05 m in identified scenarios	Abstract, Conclusion, Section V	Eq. 6; 20-scenario M_min column; safety threshold from literature
P2 — Critical	S_avg reduction across all scenarios (statistically significant, p < 0.05)	Abstract, Section V-C	Paired t-test on S_avg; Eq. 5 definition

Priority	Highlight	Where to Feature	Supporting Evidence
P3 — High	Negligible compute overhead: dT% < 5% despite stability integration	Section IV, Section V-B	S_map O(1) lookup; Eq. 12
P4 — High	Admissibility preserved: S(n) in g(n) only — h(n) unchanged	Section IV, Discussion	Formal proof Day 20; Eq. 10–11
P5 — Medium	Payload scalability: no parameter changes needed for mp = 3–9 kg	Section IV, Conclusion	CoM model Eq. 4; α tuning Day 7
P6 — Medium	Backwards compatibility: α = 0 recovers Standard A* exactly	Section IV novelty statement	Mathematical equivalence; regression test
P7 — Supporting	Path divergence D = True correlates with mp ≥ 5 kg	Section V-D	D flag in 20-scenario matrix; α = 1.0 threshold

3.2 Abstract Formulation Guide

The Abstract should follow this structure to maximise clarity and impact:

- Sentence 1–2 (Problem): Quadruped robots carrying payloads face CoM displacement risk that standard path planners ignore.
- Sentence 3–4 (Method): Load-Aware A* modifies the A* cost function with a continuous stability penalty S(n), accumulated in g(n) to preserve heuristic admissibility.
- Sentence 5–6 (Key Results): Across 20 scenarios, Load-Aware A* achieves [S_avg improvement %] reduction in mean stability penalty and [M_min improvement] increase in minimum stability margin with less than 5% computational overhead.
- Sentence 7 (Conclusion): The algorithm scales across payload masses 3–9 kg without parameter re-tuning, providing a practical stability-aware planning solution for payload-carrying quadrupeds.

Critical instruction for Abstract: Do NOT use the word ‘optimal’ without qualification. Always write ‘optimal with respect to the combined stability-cost objective’ or ‘stability-cost optimal’ — never simply ‘optimal path’, which implies geometric optimality.

3.3 What NOT to Over-Claim

The following statements would be technically inaccurate and must be avoided in the paper:

Incorrect Claim	Why It Is Wrong	Correct Version
‘Load-Aware A* always finds a shorter path’	It often finds a longer geometric path (dL ≥ 1) — shorter in stability cost, not steps	‘Load-Aware A* finds a stability-cost optimal path, which may be geometrically longer’
‘The robot is safe on all cells’	S(n) is continuous; the robot may still traverse low-M cells when no better route exists	‘The robot preferentially avoids high-instability cells; M_min is maximised across the path’

Incorrect Claim	Why It Is Wrong	Correct Version
‘Standard A* is unsafe’	Standard A* is geometrically optimal; it is stability-unaware, not inherently unsafe	‘Standard A* does not account for payload-induced CoM displacement’
‘α = 1.0 is the globally optimal weight’	α was empirically selected on 10 tuning scenarios, not proven globally optimal	‘α = 1.0 is the empirically validated minimum weight producing consistent path divergence’

4. Data Interpretation Review

This section provides a framework for interpreting the 20-scenario results table at a deeper level than metric-by-metric comparison. Three analytical lenses are applied: payload-mass stratification, stability gradient interaction, and efficiency frontier analysis.

4.1 Payload-Mass Stratification

Results should be analysed in four payload bands. The expected pattern within each band is defined below for cross-validation:

Payload Band	mp Range	Expected D Flag	Expected ΔS_avg	Expected ΔM_min	Expected dL
Band 1 — Low	3 kg	D = False (most)	Marginal (≈ 0.00–0.02)	Marginal	0 steps
Band 2 — Moderate	5 kg	D = True (most)	Moderate (0.03–0.08)	Noticeable > 0.02 m	1–2 steps
Band 3 — High	7 kg	D = True (all)	Significant (0.08–0.15)	Clear > 0.05 m	1–3 steps
Band 4 — Very High	9 kg	D = True (all)	Large (> 0.15)	Large > 0.08 m	2–3 steps

Validation check: If Band 1 (mp = 3 kg) shows D = True frequently, the implementation may have a threshold error in α. Re-confirm α = 1.0 is applied correctly. D = False at 3 kg validates the tuning.

4.2 Stability Gradient Interaction Analysis

The position of the payload within the grid (rows 3–7, cols 3–7 per experimental setup) creates a spatial S(n) gradient. The following interaction effects should be visible in the results:

- Scenarios where the optimal geometric path passes through the payload influence zone (rows 3–7): Higher D rate, larger dL, larger ΔS_avg — Load-Aware A* actively detours around the CoM-displacement zone.
- Scenarios where the geometric path avoids the payload zone naturally: D = False, dL = 0, small ΔS_avg — both algorithms find the same path because stability and length are not in conflict.
- Scenarios with high obstacle density (40–45%): Fewer routing alternatives available; Load-Aware A* may be forced through moderate-S cells, resulting in smaller ΔS_avg than lower-density equivalents.

- Scenarios with low obstacle density (20–25%): More routing alternatives; Load-Aware A* has greater freedom to select stable detours, producing larger ΔS_avg and ΔM_min.

4.3 Efficiency Frontier: The dL vs ΔM_min Trade-off

The most powerful single visualisation for the paper is a scatter plot of dL (x-axis, extra path steps) against ΔM_min (y-axis, M_min improvement in metres) across all 20 scenarios. The expected pattern defines four quadrants:

Quadrant	dL	ΔM_min	Interpretation	Paper Value
Q1 — Best Case	0	> 0	Same path length, improved margin	Strongest evidence: stability gain at zero cost
Q2 — Justified Trade-off	> 0	Large	Longer path, large margin gain	Core result: trade-off is justified
Q3 — Neutral	0	≈ 0	Same path, same margin	Low-payload baseline (mp = 3 kg)
Q4 — Inefficient	> 0	≈ 0	Longer path, negligible margin gain	Flag for individual analysis; check obstacle layout

Recommendation: If any scenario falls in Q4 (dL > 0, ΔM_min ≈ 0), this must be reported and explained — likely caused by a scenario where high-S cells are unavoidable on all routes. This is not a failure of the algorithm but of the scenario geometry, and should be discussed in the Limitations sub-section.

4.4 S_avg vs M_min Relationship

S_avg and M_min are related but measure different aspects of stability. Results must not treat them as redundant:

- S_avg (mean stability penalty): Path-level aggregate. Lower S_avg means the robot spent less cumulative time with CoM displaced. Primary effectiveness metric.
- M_min (minimum stability margin): Worst-case safety metric. A path can have low S_avg but a single dangerous cell (M < 0.05 m) that produces a low M_min. Both must be reported.
- The ideal result for Load-Aware A* is: lower S_avg AND higher M_min simultaneously. Scenarios where only one improves warrant explanation.
- If S_avg improves but M_min does not: Load-Aware A* reduced average instability but could not avoid the single worst cell. Explain this as a geometric constraint, not algorithmic failure.

5. Technical Accuracy Certification

This section constitutes Member 1's formal accuracy sign-off for Sections III, IV, and V of the paper draft. Each item has been cross-verified against Methodology v2.0 (Day 17), the algorithmic analysis (Day 18), and the theoretical justification (Day 20).

5.1 Equation Accuracy Sign-off

Eq.	Formula (must appear exactly as below)	Verified	Common Error
1	G(r,c) = { 0 if free | 1 if obstacle }	✔ **PASS**	—
2	π = < n_0, n_1, ..., n_L >; n_0 = start, n_L = goal	✔ **PASS**	Do not write n_0 = (0,0) inline — keep general
3	p_FL=(c−hs, r+hs); p_FR=(c+hs, r+hs); p_RL=(c−hs, r−hs); p_RR=(c+hs, r−hs)	✔ **PASS**	Column (c) maps to x, row (r) maps to y — do not swap
4	x_CoM = ($m^b \cdot x^b$ + m□·x_payload) / (m^b + m□)	✔ **PASS**	Must be weighted sum; not midpoint
5	S(n) = ‖x_CoM(n) − x_c(n)‖₂	✔ **PASS**	L2 norm; x_c is centroid of support polygon
6	M(n) = $\min_i$ d(x_CoM(n), $edge_i$)	✔ **PASS**	Perpendicular distance; min over all 4 edges
7	f(n) = g(n) + h(n)	✔ **PASS**	Standard A* baseline — no S(n) term
8	h(n) = |r − r_g| + |c − c_g|	✔ **PASS**	Manhattan; underscore g = goal subscript
9	g(n_next) = g(n_cur) + 1	✔ **PASS**	Uniform step cost; no stability term
10	f(n) = g(n) + α·S(n) + h(n)	✔ **PASS**	α must appear; not absorbed into S(n)
11	g(n_next) = g(n_cur) + 1 + α·S(n_next)	✔ **PASS**	n_next subscript on S; not n_cur
12	S_map = { (r,c) : S(r,c) for all (r,c) where G(r,c) = 0 }	✔ **PASS**	Free cells only; obstacle cells excluded

5.2 Algorithm Pseudocode Accuracy Check

Both algorithm pseudocodes (Standard A* and Load-Aware A*) must reflect the following mandatory elements:

Element	Standard A*	Load-Aware A*	Accuracy Status
Open list initialisation	Push (h(start), start)	Push (h(start), start)	✔ **PASS**
g-score update condition	if g_new < g_score[neighbour]	if g_new < g_score[neighbour]	✔ **PASS**
g-score calculation	g(cur) + 1	g(cur) + 1 + α·S_map[neighbour]	✔ **PASS**
f-score for heap	g_new + h(neighbour)	g_new + h(neighbour) [S(n) already in g_new]	✔ PASS — confirm S not double-counted in f
S_map reference	Not used	S_map pre-computed before loop	⚠ REVIEW — confirm S_map is built BEFORE open_list init

Element	Standard A*	Load-Aware A*	Accuracy Status
Path reconstruction	Backtrack via came_from	Identical to Standard A*	✔ **PASS**

Critical check — double-counting risk: In the f-score calculation for the heap, f = g_new + h(n). Since g_new already includes α·S(n_next) per Eq. 11, do NOT add S(n) again when computing f. If the code writes f = g_new + α·S(n) + h(n), S(n) is counted twice — this is a silent bug that inflates the penalty and alters the path.

5.3 Parameter Table Accuracy Sign-off

Parameter	Correct Value	Source	Accuracy
Grid size	R = C = 10	Methodology v2.0, III-A	✔ **PASS**
Body mass m_b	10.0 kg	Methodology v2.0, III-B	✔ **PASS**
Foot half-span hs	0.4 m	Methodology v2.0, III-B	✔ **PASS**
Cell size	1.0 m (normalised)	Methodology v2.0, III-F	✔ **PASS**
Payload range m□	3–9 kg (3, 5, 7, 9 kg)	Methodology v2.0, III-F	✔ **PASS**
Stability weight α	1.0 (empirically selected)	Day 7 tuning; Methodology v2.0	✔ **PASS**
Stability threshold	M(n) > 0.05 m	Literature-based; Methodology v2.0	✔ **PASS**
Number of scenarios	20	Methodology v2.0, III-F	✔ **PASS**
Payload position range	Rows 3–7, Cols 3–7 (interior)	Methodology v2.0, III-F	⚠ REVIEW — confirm all 20 seeds comply
Random seed validation	All 20 seeds pre-validated solvable	Day 19 matrix note	⚠ REVIEW — confirm solvability before final submission

5.4 Terminology Consistency — Final Audit

The following controlled vocabulary must be used consistently across all sections. Deviations will require corrections before submission:

Correct Term	Incorrect Variants to Remove	Section Risk
Load-Aware A*	Modified A*, Stability A*, LA-A*, load aware A*	All sections
Standard A*	Basic A*, Original A*, Normal A*, Std A*	III-C, IV, V
Stability Penalty S(n)	SP(n), Penalty(n), cost(n), instability score	III-D, IV, V

Correct Term	Incorrect Variants to Remove	Section Risk
Stability Margin M(n)	SM(n), Margin(n), safety score, buffer distance	III-B, V
S_avg	Average S, mean penalty, S_mean, avg_stability	V, VI
M_min	Minimum margin, min M, M_minimum, worst margin	V, VI
Path Divergence D	Route difference, path change, divergence flag	V
dT% (compute overhead)	Time increase %, timing overhead, speed cost	V
Admissibility	Optimality guarantee, heuristic validity (alone)	IV
CoM (Centre of Mass)	COM, center of mass, mass centre	III–VI